

## **CO 4 ACTIVITY – CORRELATION AND OUTLIERS**

### **AIM**

To perform data analysis by identifying correlations, detecting outliers, creating data groupings, analyzing statistical information, and filtering relevant data using Python libraries in Google Colab.

### **SOFTWARE REQUIREMENTS**

- Google Colab
- Python 3.x
- Pandas
- NumPy
- Matplotlib
- Seaborn
- SciPy

### **THEORY**

Data analysis is the process of inspecting, cleaning, transforming, and modeling data to discover useful information and support decision-making. It helps identify patterns, relationships, trends, and unusual observations within datasets.

The main activities performed in this experiment are:

- Identifying Correlations
- Identifying Outliers
- Creating Groupings
- Analyzing Data
- Separating and Focusing Data

These techniques help improve data quality and enable better business and research decisions.

## TASK 1: Identifying Correlations

### Objective

To identify the relationship between numerical variables in the dataset using the correlation coefficient, correlation matrix, heatmap, and scatter plot.

### Procedure

- Import the required Python libraries.
- Create or load the dataset into a Pandas DataFrame.
- Calculate the correlation matrix for numerical variables.
- Visualize the correlation matrix using a heatmap.
- Plot a scatter graph between Study Hours and Marks.
- Calculate the correlation coefficient to determine the strength of the relationship.
- Interpret the obtained results.

### Correlation Matrix

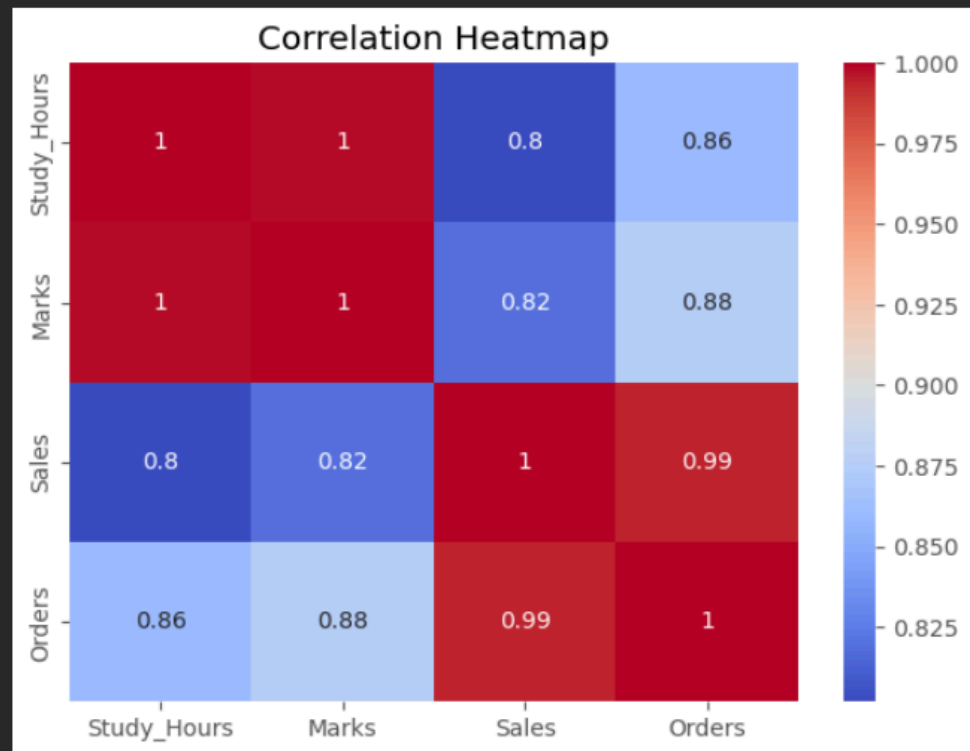
```
corr=df.corr(numeric_only=True)

print(corr)
```

|             | Study_Hours | Marks    | Sales    | Orders   |
|-------------|-------------|----------|----------|----------|
| Study_Hours | 1.000000    | 0.997897 | 0.801812 | 0.860066 |
| Marks       | 0.997897    | 1.000000 | 0.819589 | 0.875478 |
| Sales       | 0.801812    | 0.819589 | 1.000000 | 0.993840 |
| Orders      | 0.860066    | 0.875478 | 0.993840 | 1.000000 |

## Correlation Heatmap

```
plt.figure(figsize=(7,5))
sns.heatmap(corr,
            annot=True,
            cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



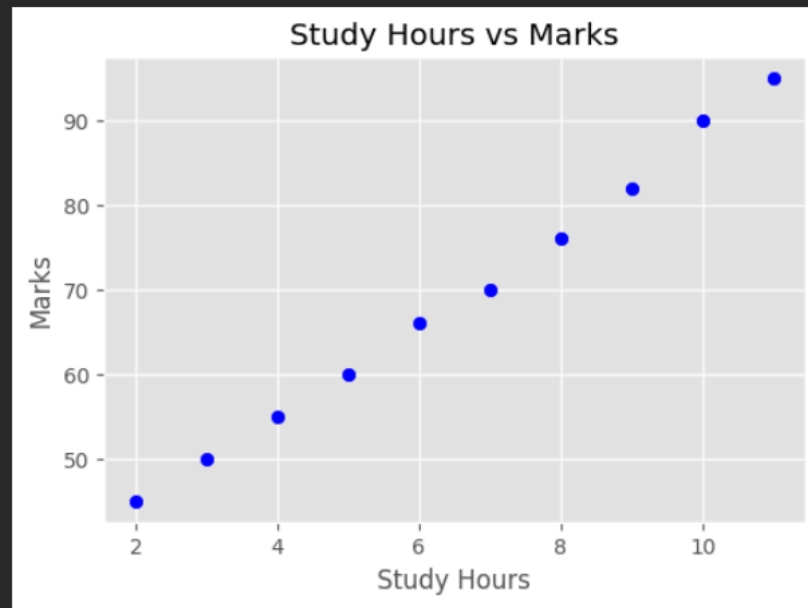
## Scatter Plot

```
plt.figure(figsize=(6,4))

plt.scatter(df["Study_Hours"],
            df["Marks"],
            color="blue")

plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Study Hours vs Marks")

plt.show()
```



## Correlation Coefficient

```
r=df["Study_Hours"].corr(df["Marks"])

print("Correlation Coefficient =",round(r,2))

Correlation Coefficient = 1.0
```

## TASK 2: Identifying Outliers

### Objective

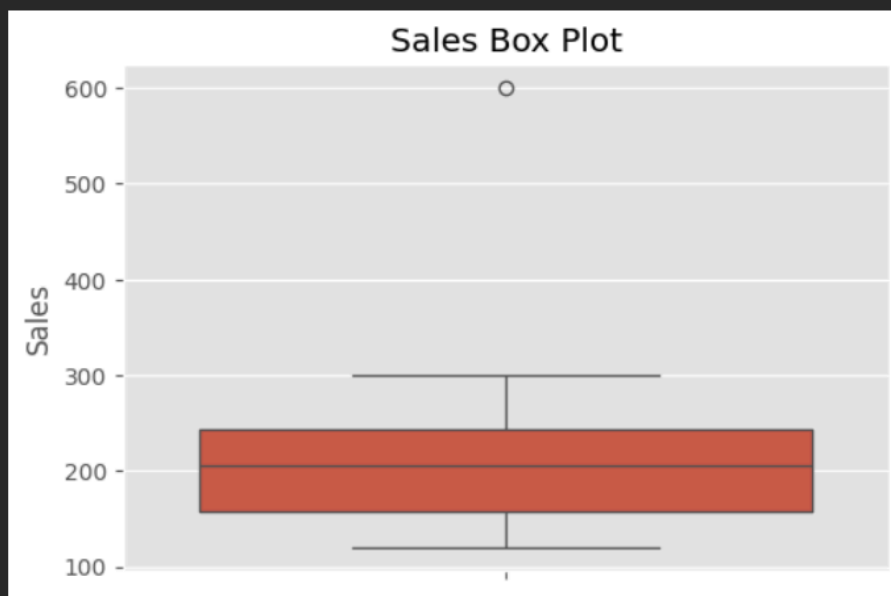
To identify unusual or abnormal observations in the dataset using Box Plot and Z-score methods.

### Procedure

- Import the required libraries.
- Select the numerical column for outlier detection.
- Generate a Box Plot to visualize outliers.
- Calculate the Z-score for each data value.
- Identify observations with high Z-score values as outliers.
- Display the detected outliers.

### Box Plot

```
plt.figure(figsize=(6,4))  
sns.boxplot(y=df["Sales"])  
plt.title("Sales Box Plot")  
plt.show()
```



## Z-Score Table

```
from scipy.stats import zscore  
  
df["Zscore"]=zscore(df["Sales"])  
  
print(df[["Sales","Zscore"]])
```

|   | Sales | Zscore    |
|---|-------|-----------|
| 0 | 120   | -0.878788 |
| 1 | 150   | -0.651515 |
| 2 | 130   | -0.803030 |
| 3 | 180   | -0.424242 |
| 4 | 200   | -0.272727 |
| 5 | 220   | -0.121212 |
| 6 | 210   | -0.196970 |
| 7 | 250   | 0.106061  |
| 8 | 300   | 0.484848  |
| 9 | 600   | 2.757576  |

## Outliers

```
outliers=df[np.abs(df["Zscore"])>2]  
  
print("Outliers")  
  
print(outliers)
```

| Outliers |             |       |       |        |        |         |          |
|----------|-------------|-------|-------|--------|--------|---------|----------|
|          | Study_Hours | Marks | Sales | Orders | Region | Product | Zscore   |
| 9        | 11          | 95    | 600   | 90     | South  | Laptop  | 2.757576 |

## TASK 3 : Creating Groupings

### Objective

To organize and summarize data into meaningful groups using GroupBy and Pivot Table operations.

### Procedure

- Load the dataset into a DataFrame.
- Group the data based on the Region column.
- Calculate the total Sales and Orders for each region.
- Create a Pivot Table to summarize sales according to region and product.
- Display the grouped results.
- Visualize the grouped data using a bar chart.

### GroupBy Region

```
group=df.groupby("Region")[["Sales","Orders"]].sum()  
  
print(group)
```

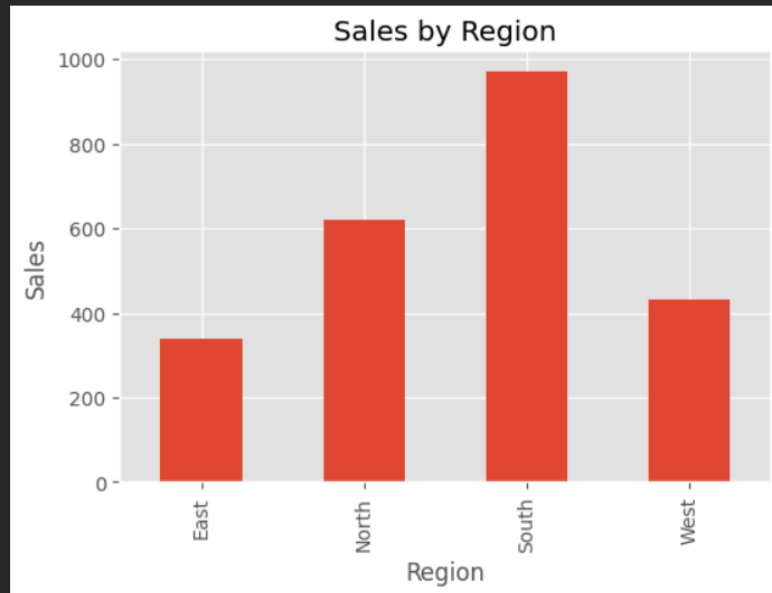
|        | Sales | Orders |
|--------|-------|--------|
| Region |       |        |
| East   | 340   | 69     |
| North  | 620   | 120    |
| South  | 970   | 162    |
| West   | 430   | 83     |

## Sales by Region Bar Chart

```
group["Sales"].plot(kind="bar",figsize=(6,4))

plt.title("Sales by Region")
plt.ylabel("Sales")

plt.show()
```



## Pivot Table

```
pivot=pd.pivot_table(df,
                      values="Sales",
                      index="Region",
                      columns="Product",
                      aggfunc="sum",
                      fill_value=0)
```

pivot

| Product | Laptop | Mobile | Monitor | Tablet |
|---------|--------|--------|---------|--------|
| Region  |        |        |         |        |
| East    | 210    | 0      | 0       | 130    |
| North   | 120    | 0      | 300     | 200    |
| South   | 600    | 150    | 220     | 0      |
| West    | 180    | 250    | 0       | 0      |

## TASK 4 : Analyze Data

### Objective

To analyze the dataset using descriptive statistics and graphical visualization to understand trends and patterns.

### Procedure

- Calculate descriptive statistics for the numerical columns.
- Determine the mean, median, and mode of the Sales column.
- Generate a line graph to visualize the monthly sales trend.
- Analyze the statistical results.
- Interpret the observed trend.

### Descriptive Statistics

```
print(df.describe())
```

|       | Study_Hours | Marks     | Sales      | Orders    | Zscore    |
|-------|-------------|-----------|------------|-----------|-----------|
| count | 10.00000    | 10.000000 | 10.000000  | 10.000000 | 10.000000 |
| mean  | 6.50000     | 68.900000 | 236.000000 | 43.400000 | 0.000000  |
| std   | 3.02765     | 16.861857 | 139.140217 | 18.774687 | 1.054093  |
| min   | 2.00000     | 45.000000 | 120.000000 | 25.000000 | -0.878788 |
| 25%   | 4.25000     | 56.250000 | 157.500000 | 31.250000 | -0.594697 |
| 50%   | 6.50000     | 68.000000 | 205.000000 | 40.500000 | -0.234848 |
| 75%   | 8.75000     | 80.500000 | 242.500000 | 46.500000 | 0.049242  |
| max   | 11.00000    | 95.000000 | 600.000000 | 90.000000 | 2.757576  |

## Monthly Sales Trend

```
months=["Jan","Feb","Mar","Apr","May","Jun","Jul","Aug","Sep","Oct"]

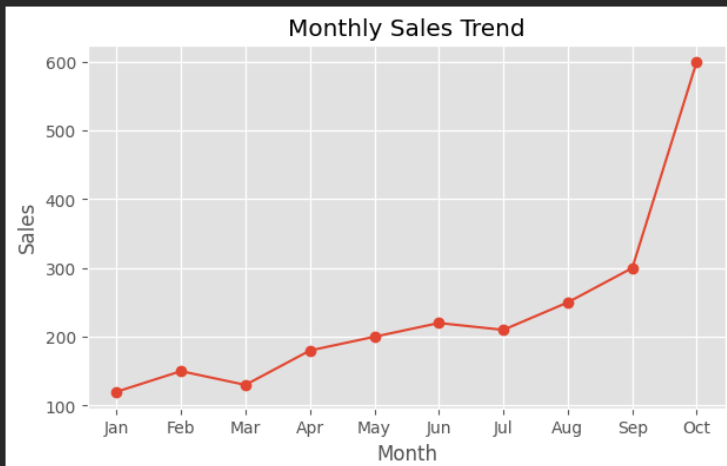
plt.figure(figsize=(7,4))

plt.plot(months,
         df["Sales"],
         marker="o")

plt.title("Monthly Sales Trend")

plt.xlabel("Month")
plt.ylabel("Sales")

plt.show()
```



## Mean, Median and Mode

```
print("Mean =",df["Sales"].mean())

print("Median =",df["Sales"].median())

print("Mode =")
print(df["Sales"].mode())
```

Mean = 236.0

Median = 205.0

Mode =

0 120

1 130

2 150

3 180

4 200

5 210

6 220

7 250

8 300

9 600

Name: Sales, dtype: int64

## TASK 5: Separating and Focusing the Data

### Objective

To filter, sort, and clean the dataset by selecting relevant records and handling missing values for effective analysis.

### Procedure

- Filter the dataset to display only high-sales records.
- Sort the dataset based on sales values.
- Select the required columns for analysis.
- Apply date-based filtering where necessary.
- Introduce and handle missing values.
- Display the cleaned and filtered dataset.

### Filtered Data

```
high_sales=df[df["Sales"]>150]
```

```
high_sales
```

|   | Study_Hours | Marks | Sales | Orders | Region | Product | Zscore    |
|---|-------------|-------|-------|--------|--------|---------|-----------|
| 3 | 5           | 60    | 180   | 35     | West   | Laptop  | -0.424242 |
| 4 | 6           | 66    | 200   | 40     | North  | Tablet  | -0.272727 |
| 5 | 7           | 70    | 220   | 42     | South  | Monitor | -0.121212 |
| 6 | 8           | 76    | 210   | 41     | East   | Laptop  | -0.196970 |
| 7 | 9           | 82    | 250   | 48     | West   | Mobile  | 0.106061  |
| 8 | 10          | 90    | 300   | 55     | North  | Monitor | 0.484848  |
| 9 | 11          | 95    | 600   | 90     | South  | Laptop  | 2.757576  |

## Sorted Data

```
sorted_df=df.sort_values(by="Sales",ascending=False)
```

```
sorted_df
```

|   | Study_Hours | Marks | Sales | Orders | Region | Product | Zscore    |
|---|-------------|-------|-------|--------|--------|---------|-----------|
| 9 | 11          | 95    | 600   | 90     | South  | Laptop  | 2.757576  |
| 8 | 10          | 90    | 300   | 55     | North  | Monitor | 0.484848  |
| 7 | 9           | 82    | 250   | 48     | West   | Mobile  | 0.106061  |
| 5 | 7           | 70    | 220   | 42     | South  | Monitor | -0.121212 |
| 6 | 8           | 76    | 210   | 41     | East   | Laptop  | -0.196970 |
| 4 | 6           | 66    | 200   | 40     | North  | Tablet  | -0.272727 |
| 3 | 5           | 60    | 180   | 35     | West   | Laptop  | -0.424242 |
| 1 | 3           | 50    | 150   | 30     | South  | Mobile  | -0.651515 |
| 2 | 4           | 55    | 130   | 28     | East   | Tablet  | -0.803030 |
| 0 | 2           | 45    | 120   | 25     | North  | Laptop  | -0.878788 |

## Selected Columns

```
selected=df[["Product","Region","Sales"]]
```

selected

|   | Product | Region | Sales |
|---|---------|--------|-------|
| 0 | Laptop  | North  | 120   |
| 1 | Mobile  | South  | 150   |
| 2 | Tablet  | East   | 130   |
| 3 | Laptop  | West   | 180   |
| 4 | Tablet  | North  | 200   |
| 5 | Monitor | South  | 220   |
| 6 | Laptop  | East   | 210   |
| 7 | Mobile  | West   | 250   |
| 8 | Monitor | North  | 300   |
| 9 | Laptop  | South  | 600   |

## Date Filtering

```
df["Date"]=pd.date_range(start="2026-01-01",  
                          periods=len(df))
```

```
filtered=df[df["Date"]>"2026-01-05"]
```

filtered

|   | Study_Hours | Marks | Sales | Orders | Region | Product | Zscore    | Date       |
|---|-------------|-------|-------|--------|--------|---------|-----------|------------|
| 5 | 7           | 70    | 220   | 42     | South  | Monitor | -0.121212 | 2026-01-06 |
| 6 | 8           | 76    | 210   | 41     | East   | Laptop  | -0.196970 | 2026-01-07 |
| 7 | 9           | 82    | 250   | 48     | West   | Mobile  | 0.106061  | 2026-01-08 |
| 8 | 10          | 90    | 300   | 55     | North  | Monitor | 0.484848  | 2026-01-09 |
| 9 | 11          | 95    | 600   | 90     | South  | Laptop  | 2.757576  | 2026-01-10 |

## Missing Value Handling

```
df.loc[2,"Sales"]=np.nan  
  
print(df.isnull().sum())
```

```
Study_Hours    0  
Marks          0  
Sales          1  
Orders         0  
Region         0  
Product        0  
Zscore         0  
Date           0  
dtype: int64
```

## Final Dataset

```
df["Sales"]=df["Sales"].fillna(df["Sales"].mean())  
  
print(df)
```

|   | Study_Hours | Marks | Sales      | Orders | Region | Product | Zscore    | Date       |
|---|-------------|-------|------------|--------|--------|---------|-----------|------------|
| 0 | 2           | 45    | 120.000000 | 25     | North  | Laptop  | -0.878788 | 2026-01-01 |
| 1 | 3           | 50    | 150.000000 | 30     | South  | Mobile  | -0.651515 | 2026-01-02 |
| 2 | 4           | 55    | 247.777778 | 28     | East   | Tablet  | -0.803030 | 2026-01-03 |
| 3 | 5           | 60    | 180.000000 | 35     | West   | Laptop  | -0.424242 | 2026-01-04 |
| 4 | 6           | 66    | 200.000000 | 40     | North  | Tablet  | -0.272727 | 2026-01-05 |
| 5 | 7           | 70    | 220.000000 | 42     | South  | Monitor | -0.121212 | 2026-01-06 |
| 6 | 8           | 76    | 210.000000 | 41     | East   | Laptop  | -0.196970 | 2026-01-07 |
| 7 | 9           | 82    | 250.000000 | 48     | West   | Mobile  | 0.106061  | 2026-01-08 |
| 8 | 10          | 90    | 300.000000 | 55     | North  | Monitor | 0.484848  | 2026-01-09 |
| 9 | 11          | 95    | 600.000000 | 90     | South  | Laptop  | 2.757576  | 2026-01-10 |

## **OBSERVATIONS**

### **Correlation Analysis**

- Study Hours and Marks show a strong positive correlation.
- Correlation coefficient is close to +1, indicating that marks increase as study hours increase.

### **Outlier Detection**

- One sales value is identified as an outlier.
- The Box Plot and Z-Score method successfully detect the abnormal observation.

### **Grouping**

- Sales are grouped according to regions.
- Pivot Tables summarize sales across different products.

### **Statistical Analysis**

- Mean, median, and mode describe the distribution of sales.
- Descriptive statistics provide insights into data spread and central tendency.

### **Data Filtering**

- High-value sales records are extracted.
- Sorting organizes the dataset efficiently.
- Missing values are replaced with the mean value.

## **RESULT**

The experiment was successfully completed using Python in Google Colab. Correlation analysis identified relationships among variables, outlier detection highlighted abnormal observations, grouping summarized categorical data, descriptive statistics provided meaningful insights, and filtering techniques improved data quality. These methods collectively demonstrate the essential steps involved in effective data analysis.